HERBAL KNOWLEDGE INTELLIGENCE SYSTEM: A BILINGUAL RETRIEVAL-AUGMENTED GENERATION ASSISTANT FOR SRI LANKAN MEDICINAL PLANTS

INDIVIDUAL THESIS

Munasinghe M P

(IT22640598)

Individual component: Herbal Knowledge Intelligence System

Group ID: R26-IT-021 (Intelligent Ayurvedic Herbal Plant Identification and Economic Analysis in Sri Lanka)

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) Degree in Information Technology

Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Munasinghe M P

Student ID: IT22640598

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr. Nelum Amarasena		
Co-supervisor: Ms. Kaushika Kahatapitiya		

ABSTRACT

Sri Lanka's Ayurvedic tradition relies on knowledge of about 1,400 medicinal plant species, but this knowledge is held mainly by elderly practitioners and is hard to access, especially in Sinhala. Existing tools are ontologies that need SPARQL expertise or pattern-matching chatbots with shallow knowledge, and general language models can fabricate health information. This thesis presents the Herbal Knowledge Intelligence System (HKAS), the conversational component of research project R26-IT-021, "Intelligent Ayurvedic Herbal Plant Identification and Economic Analysis in Sri Lanka".

HKAS is a bilingual (English and Sinhala) Retrieval-Augmented Generation assistant built with LangChain, ChromaDB, OpenAI text-embedding-3-large and gpt-4o-mini, served by Flask to a Flutter client. Beyond a baseline pipeline it adds single-call query normalisation (context resolution, Sinhala-to-English translation, multi-plant splitting), evidence gating by cosine distance with a Sinhala lexical guard, flexible audience categories (explicit, detected or page-based, up to two combined), and prompts that require precautions and forbid invented references.

A bilingual paired set of 12 queries and an eight-test interaction suite were used. The correct plant entry was retrieved for 5 of 6 English and 6 of 6 Sinhala questions, no answerable Sinhala question was refused, and all eight interaction tests returned the expected categories, including multi-category answers, per-message switching and explicit overrides. The evaluation also identified refinement areas: Sinhala phrasing, an overlap between plant and disease entries in retrieval, and prompt-label leaks. Recommendations are to address these, add systematic out-of-knowledge-base tests and obtain expert validation.

Keywords: Retrieval-Augmented Generation, Medicinal Plants, Ayurveda, Sinhala Language Processing, Conversational AI

ACKNOWLEDGEMENTS

I sincerely thank my supervisor, Mr. Nelum Amarasena, and my co-supervisor, Ms. Kaushika Kahatapitiya, for their guidance, constructive criticism and steady support throughout this research. Their feedback on novelty and measurability directly shaped the evaluation design reported in this thesis.

I am grateful to the panel members who reviewed the project at each stage, and to my teammates in research group R26-IT-021, whose components of plant identification, explainable quality assessment and cultivation recommendation make up the wider system into which this work fits. I also thank the Department of Information Technology of the Sri Lanka Institute of Information Technology for the facilities and resources provided.

Finally, I thank my family and friends for their encouragement during the final year of my degree, and the users who tried the system and shared their comments.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Overall Project Context 1

1.2 Individual Component Context 2

1.3 Problem Statement 2

1.4 Research Gap 3

1.5 Aim and Objectives 3

1.6 Research Questions or Hypotheses 4

1.7 Scope Limitations and Assumptions 5

1.8 Individual Contribution 5

1.9 Thesis Structure 6

2 LITERATURE REVIEW AND RELATED WORK 7

2.1 Review Method and Scope 7

2.2 Domain and Theoretical Foundations 7

2.2.1 Language models and the transformer 7

2.2.2 Hallucination and the case for grounding 7

2.2.3 Embeddings, dense retrieval and vector stores 8

2.2.4 Multilingual and low-resource language processing 8

2.2.5 Evaluating RAG systems 8

2.3 Related Research 9

2.3.1 Knowledge systems for Sri Lankan medicinal plants 9

2.3.2 Language models in health and education 9

2.3.3 Knowledge base construction with RAG 9

2.4 Existing Systems Products and Techniques 10

2.5 Gap Synthesis and Conceptual Framework 10

3 RESEARCH METHODOLOGY 12
3.1 Research Design 12
3.2 Data Participants or Study Materials 12
3.2.1 Knowledge base 12
3.2.2 Evaluation sets 13
3.2.3 Participants 13
3.3 Experimental or Study Procedure 13
3.4 Measures and Evaluation Metrics 14
3.5 Data Analysis 15
3.6 Ethics Privacy Security and Responsible AI 15
3.7 Reproducibility and Research Artefacts 16
4 PRODUCT AND SOLUTION DESIGN 17
4.1 Requirements 17
4.2 Component Architecture within the Complete Solution 18
4.3 Design Alternatives and Decisions 18
4.4 Detailed Design 19
4.4.1 Knowledge base schema 19
4.4.2 Ingestion pipeline 20
4.4.3 Query pipeline and evidence gating 20
4.4.4 Audience resolution 21
4.4.5 Prompt design 22
4.4.6 API and client design 23
4.5 Standards and Professional Practice 23
5 IMPLEMENTATION INTEGRATION AND TESTING 25
5.1 Development Environment and Technology Stack 25
5.2 Component Implementation 26

5.2.1 Knowledge base and ingestion 26
5.2.2 Query pipeline, gating and audience resolution 26
5.2.3 Persona-aware answer generation 27
5.2.4 API and client 28
5.3 Verification and Software Testing 28
5.4 Deployment and Operation 29
6 RESULTS AND VALIDATION 30
6.1 Research Results 30
6.2 Product and Technical Results 32
6.3 User or Stakeholder Validation 32
6.4 Integration Results 32
6.5 Summary of Findings 33
7 DISCUSSION 34
7.1 Interpretation of Findings 34
7.2 Comparison with Literature Baselines and Existing Solutions 34
7.3 Research Contribution 35
7.4 Product Contribution 35
7.5 Limitations and Threats to Validity 35
7.6 Implications 36
8 CONCLUSION AND FUTURE DIRECTIONS 37
8.1 Conclusion 37
8.2 Achievement of Objectives 37
8.3 Individual Research and Product Contributions 37
8.4 Future Directions 38
REFERENCES 39
GLOSSARY 45

APPENDICES 46

Appendix A Research Instruments or Extended Results 46

Appendix B Academic Integrity and AI Use Disclosure 46

LIST OF FIGURES

Figure 2.1 Conceptual framework linking observed problems to HKAS design principles 11
Figure 3.1 Research and development methodology 12
Figure 3.2 Evaluation procedure 14
Figure 4.1 HKAS component architecture and interfaces 18
Figure 4.2 Ingestion pipeline 20
Figure 4.3 Query pipeline with evidence gating and the Sinhala guard 21
Figure 4.4 Resolution of audience categories for one message 22

LIST OF TABLES

Table 1.1 Components of research project R26-IT-021 1
Table 1.2 Individual contribution compared with group work 5
Table 2.1 Comparison of existing approaches and HKAS 10
Table 3.1 Evaluation measures 14
Table 4.1 Functional requirements 17
Table 4.2 Non-functional requirements 17
Table 4.3 Design decisions and alternatives 19
Table 4.4 Knowledge base entry fields 20
Table 4.5 Audience categories and response style 22
Table 5.1 Technology stack 25
Table 5.2 Configuration parameters 25
Table 5.3 Representative test cases 28
Table 6.1 Evaluation suites 30
Table 6.2 Outcome by language, Suite A 30
Table 6.3 Per-query results, Suite A 31
Table 6.4 Delivered functionality 32

Table A 1 Suite A: question pairs 46
Table A 2 Suite B: flexible-interaction tests 46

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CNN	Convolutional Neural Network
Grad-CAM	Gradient-weighted Class Activation Mapping
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
KB	Knowledge Base
LLM	Large Language Model
NFR	Non-Functional Requirement
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SDG	Sustainable Development Goal
SPARQL	SPARQL Protocol and RDF Query Language
SUS	System Usability Scale
UI	User Interface

1 INTRODUCTION

1.1 Overall Project Context

Sri Lanka has a medicinal plant heritage of remarkable depth. The Department of Ayurveda and the wider literature commonly cite around 1,400 plant species used in traditional medicine [1], and Ayurveda itself has been practiced on the island for several thousand years. Plants such as Kohomba (neem), Komarika (aloe vera), Aralu, Bulu, Nelli and Venivel appear throughout traditional formulations and remain in everyday use. The World Health Organization has noted the continuing global importance of traditional and complementary medicine and the need for reliable, safe information about it [3].

This thesis is an individual component of the final-year research project R26-IT-021, titled "Intelligent Ayurvedic Herbal Plant Identification and Economic Analysis in Sri Lanka". The project is an integrated, mobile-first ecosystem built from four tightly coupled components that share a common medicinal plant knowledge base and a Flutter-based mobile front end, as summarised in Table 1.1 and Figure 1.1. A user can photograph a plant, learn its identity, see an explainable assessment of its quality, ask questions about its traditional uses in English or Sinhala, and, in the case of farmers, receive cultivation and economic recommendations.

Table 1.1 Components of research project R26-IT-021

Student ID	Component	Main technique
IT22640598	Herbal Knowledge Intelligence System	Bilingual RAG conversational assistant
IT22168986	Mobile-based detection of Sri Lankan Ayurvedic plants	CNN image recognition
IT22112132	Explainable AI system for plant quality assessment	ResNet50 with Grad-CAM
IT22132796	AI-driven cultivation recommendation for small-scale farmers	Machine learning recommendation and yield prediction

The components are designed to interlock. The plant identification module produces a plant name, which the knowledge component receives as conversational context so that the user can ask follow-up questions without retyping it. The knowledge

component therefore acts as the explanatory and educational layer of the whole ecosystem, turning a bare classification label into usable, trustworthy information.

1.2 Individual Component Context

The individual component reported here is the Herbal Knowledge Intelligence System, whose conversational assistant is referred to as the Herbal Knowledge Assistant System (HKAS). Its purpose is to answer natural-language questions about Sri Lankan medicinal plants (for example, "What is neem used for?" or the equivalent Sinhala question) with answers that are grounded in a curated knowledge base, expressed in the language of the question, and adapted to the kind of user asking.

HKAS is built on Retrieval-Augmented Generation (RAG), in which a retriever selects relevant passages from an external knowledge base and a generator, a large language model, composes an answer from them [4], [5]. The approach keeps answers tied to a maintained knowledge source and lets the knowledge be updated without retraining a model. The implementation uses Python, LangChain [6], ChromaDB [7], the OpenAI text-embedding-3-large embedding model [8] and the gpt-4o-mini chat model [9], behind a Flask [10] API consumed by a Flutter [11] client.

1.3 Problem Statement

Four connected problems motivate this work.

Erosion of traditional knowledge. Knowledge of plant names, parts used, preparations and applications has been transmitted orally or through scarce literature and is concentrated among practitioners who are now elderly. As modern lifestyles displace traditional practice, the transfer to younger generations is weak, and the knowledge is at risk of being lost.

Poor accessibility. Reliable information is scattered across costly printed books, research papers and web pages of uneven quality, and cannot easily be searched. Most online material is in English, whereas a large part of the population communicates mainly in Sinhala.

Inadequate digital tools. Existing systems are polarised. Ontologies hold deep, structured knowledge but require SPARQL query skills [12]. Chatbots offer an easy interface but rely on pattern matching or fixed intents over shallow knowledge [13], [14], so they fail on questions phrased differently from the stored patterns.

Risk of unreliable generation. General-purpose LLMs converse fluently but can produce plausible, unsupported statements, known as hallucinations [15]. In a health-related domain, a fabricated dose, preparation or contraindication can cause real harm [16], [17]. A system that answers everything, including what it does not know, is therefore unsuitable.

The problem addressed by this thesis is thus how to give the public, students, practitioners and researchers conversational, bilingual access to Sri Lankan herbal knowledge that is deep, grounded and safe, and how to demonstrate that quality through measurement rather than assertion.

1.4 Research Gap

The review in Chapter 2 identifies five gaps that this work addresses. There is no RAG-based assistant for Sri Lankan herbal medicine: RAG has been applied to education [18], [19], healthcare [20], [21], agriculture [22] and knowledge management [23], but not to Sri Lankan medicinal plants. Semantic understanding of Sinhala in this domain remains limited; existing Sinhala herbal chatbots rely on AIML patterns or intent classification [13], [14], and Sinhala is a low-resource language in NLP [24]. Neither herbal systems nor typical RAG prototypes decide, in an inspectable way, when retrieved evidence is too weak to answer [25]. Adaptive assistants usually lock the user into one category, whereas the project panel asked for dynamic switching and multi-category answers. No benchmark or agreed metric set exists for bilingual herbal assistants; this thesis proposes and applies one, drawing on RAG evaluation practice [26].

1.5 Aim and Objectives

Aim. To design, implement and evaluate a bilingual (English and Sinhala) Retrieval-Augmented Generation assistant that provides accurate, grounded and safe

information about Sri Lankan Ayurvedic medicinal plants, adapts flexibly to different users, and integrates with the other components of project R26-IT-021.

RO1: Build and curate a structured knowledge base of Sri Lankan medicinal plants, with names, uses, preparation, precautions and evidence fields.

RO2: Implement a RAG pipeline with entry-aware ingestion, dense retrieval and LLM-based answer generation.

RO3: Implement evidence gating so that the system declines to answer when retrieved evidence is insufficient.

RO4: Support Sinhala and English questions through query normalization that translates Sinhala questions into English search queries.

RO5: Provide flexible audience categories (four types) with explicit selection, per-message switching and multi-category answers, and support questions about several plants.

RO6: Evaluate the system with a bilingual paired set and a flexible-interaction suite, and analyze the defects found.

RO7: Accept plant names and disease labels from the other project components as questions to be explained.

1.6 Research Questions or Hypotheses

RQ1. Can a RAG pipeline over a curated knowledge base give accurate and grounded answers to questions about Sri Lankan medicinal plants?

RQ2. How does the evidence gate behave: does it refuse unsupported questions without wrongly refusing answerable ones?

RQ3. How well does English-pivot query normalization handle Sinhala questions compared with English questions?

RQ4. Can flexible audience resolution route single-category, switched, multi-category and explicitly chosen requests correctly, and answer multi-plant questions without mixing facts?

Given the size of the evaluation sets (Section 7.5), the research questions are answered descriptively, by counting outcomes and inspecting individual cases, and not by statistical testing.

1.7 Scope Limitations and Assumptions

In scope are text-based question answering in English and Sinhala over a curated set of Sri Lankan medicinal plants; the RAG pipeline, evidence gate, query normalization and audience categories; the Flask API and the Flutter chat and persona interface; and the evaluation described in Chapter 3. Out of scope are image-based plant identification, quality assessment and cultivation recommendation (other group members); fine-tuning or training of language models; voice interaction; clinical diagnosis or treatment advice; and production-scale cloud deployment. Assumptions are that the knowledge base content is accurate to the cited sources; the OpenAI services are available when the system runs; users read the educational disclaimer; and the evaluation questions represent typical user queries. Limitations are that the knowledge base covers a limited number of plants, and the assistant is a decision-support and educational tool, not a medical device.

1.8 Individual Contribution

Table 1.2 separates the contribution of this thesis from the shared work of the group.

Table 1.2 Individual contribution compared with group work

Area	Individual contribution (this thesis)	Shared or other members
Knowledge base	Plant entries with precaution and evidence fields; ingestion by entry	disease_knowledge.txt supports the quality assessment component
RAG engine	Query normalisation, retrieval, distance gate, Sinhala guard, prompt rules, audience resolution	None
Languages	Bilingual handling through English-pivot query normalisation	None
Backend and client	Flask API for the assistant; Flutter chat, persona chips and pages	Overall mobile shell shared with the group
Evaluation	evaluate_accuracy.py, scoring sheets, analysis	None
Integration	Short-label handling for plant names and disease labels	Plant quality outputs

1.9 Thesis Structure

Chapter 2 reviews the literature and related systems and derives the research gap. Chapter 3 describes the research methodology, data and evaluation design. Chapter 4 presents the requirements and the design of the solution, and Chapter 5 its implementation, integration and testing. Chapter 6 reports the results and validation, and Chapter 7 discusses their meaning, contributions and limitations. Chapter 8 concludes and outlines future directions. The references, glossary and appendices follow.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The review covers three bodies of work: digital systems for Sri Lankan and Ayurvedic medicinal plant knowledge; language-model and RAG technology and its application to health and education; and methods for evaluating RAG systems. Sources were gathered from IEEE Xplore, the ACM Digital Library, Springer, Elsevier, PubMed and Google Scholar using combinations of the terms "medicinal plants", "Ayurveda", "Sri Lanka", "ontology", "chatbot", "Sinhala", "retrieval-augmented generation", "hallucination" and "multilingual embeddings". Priority was given to peer-reviewed work from 2015 onwards, with foundational papers included irrespective of date.

Studies were included if they addressed herbal or Ayurvedic knowledge systems, conversational agents in health or education, retrieval or generation methods directly relevant to the pipeline, or evaluation of RAG. Studies were excluded if they concerned image-only plant classification, which belongs to other components of the project.

2.2 Domain and Theoretical Foundations

2.2.1 Language models and the transformer

Modern language models are built on the transformer architecture, whose self-attention mechanism models long-range dependencies efficiently [27]. Pre-trained encoders such as BERT [28] established transfer learning for NLP, while very large generative models such as GPT-3 showed that few-shot prompting can perform new tasks without retraining [29]. Open models such as Llama 2 [30] offer alternatives to proprietary services. Surveys describe how these models are trained, aligned and applied [31].

2.2.2 Hallucination and the case for grounding

A well-documented weakness of generative models is hallucination, in which output is fluent but not supported by the source material or by fact [15]. Large models

encode substantial clinical knowledge yet remain unreliable without safeguards [16], [17]. Retrieval-Augmented Generation mitigates the problem by conditioning generation on passages fetched from an external knowledge source at query time [4], and a growing body of work catalogues RAG variants and their design choices [5].

2.2.3 Embeddings, dense retrieval and vector stores

Dense retrieval represents queries and passages as vectors and ranks passages by similarity, often outperforming purely lexical matching on semantic queries [32]. Sentence-level encoders produce embeddings suited to semantic search [33], and contrastively pre-trained embedding models are now offered as services [8], [34]. Vector databases such as ChromaDB [7] store the vectors and return nearest neighbours with a distance score. That score is the quantity that HKAS uses to decide whether evidence is sufficient.

2.2.4 Multilingual and low-resource language processing

Cross-lingual models such as XLM-R learn shared representations across many languages [35], and multilingual embedding services embed queries in different languages into a common space. Nevertheless, performance on lower-resource languages is typically weaker than on English [24]. Sinhala has far fewer corpora and tools than English, so a pipeline that depends entirely on Sinhala embeddings can retrieve poorly, which motivates the English-pivot query normalization used in this work.

2.2.5 Evaluating RAG systems

RAG evaluation separates the quality of retrieval from the quality of generation. The RAGAS framework defines reference-light metrics such as faithfulness (are the claims in the answer supported by the retrieved context), answer relevancy, and context precision and recall [26]. Studies of RAG engineering identify recurring failure points, including missing content, poor ranking, and answers that ignore the retrieved context [25]. Models also use long contexts unevenly [36], which argues for concise, well-bounded retrieved context.

2.3 Related Research

2.3.1 Knowledge systems for Sri Lankan medicinal plants

Jeyananthan et al. built a Protege ontology of Sri Lankan medicinal plants, with classes for plants, plant parts, diseases, preparation methods, mixtures and application categories [12]. The ontology is structured and deep, but querying it requires SPARQL, which excludes most users. Extending it with new plants or attributes is a manual task.

Jayalath et al. developed an Ayurvedic knowledge sharing platform that combined plant identification with a Sinhala virtual assistant based on AIML pattern matching [13]. The assistant answers only questions that match predefined patterns. Senevirathne et al. produced a mobile tool combining a Faster R-CNN plant identifier, a 3D viewer and a Sinhala AIML chatbot with intent classification [14]. Its chatbot reaches high accuracy on its predefined intents, but its knowledge depth is limited to what was manually encoded.

2.3.2 Language models in health and education

Abd-alrazaq et al. discuss the opportunities and challenges of LLMs in medical education, including accuracy and hallucination [37], and Thirunavukarasu et al. review LLMs in medicine [17]. RAG-based assistants have been proposed for education [18], assessment [19], behavioral therapy [20], medical clinics [21], system configuration [38] and irrigation advice [22]. These works show that RAG is a credible foundation for domain assistants, but none targets Sri Lankan herbal medicine or Sinhala.

2.3.3 Knowledge base construction with RAG

Pondel et al. examine how to build a knowledge base for an LLM and RAG assistant, including choices about chunking and embeddings [23]. Their guidance is relevant to this work, in which chunking is a first-order design decision (Section 4.3).

2.4 Existing Systems Products and Techniques

Table 2.1 compares the principal existing approaches with the proposed system. General-purpose chat assistants answer fluently in many languages but are ungrounded in any curated Sri Lankan source and offer no refusal behaviour tied to evidence. Plant identification apps identify species from photographs using community image data [39], but do not provide Ayurvedic preparation, precaution or Sinhala-language knowledge.

Table 2.1 Comparison of existing approaches and HKAS

Aspect	Ontology [12]	Pattern chatbot [13]	Intent chatbot [14]	**HKAS (proposed)**
Natural-language interface	No	Yes	Yes	Yes
Handles novel phrasing	No	No	Limited	Yes (semantic retrieval)
Knowledge depth	High	Low	Low	High (curated entries)
Sinhala support	No	Pattern-based	Limited	English-pivot query normalisation
Updating knowledge	Manual	Manual	Manual	Add entries and re-ingest
Answer grounding	Structured data	Patterns	Patterns	Retrieved entries
Refuses when unsupported	Not applicable	No	No	Yes (distance gate, Sinhala guard)
User-type adaptation	No	No	No	Yes (four categories, switchable)
Several plants in one question	No	No	No	Yes (up to three)
Link to other components	No	Yes	Yes	Yes (plant and disease labels)

2.5 Gap Synthesis and Conceptual Framework

The literature shows a clear pattern. Knowledge-rich systems are hard to use, easy-to-use systems are knowledge-poor, and modern RAG techniques that could bridge the two have not been applied to Sri Lankan herbal medicine in Sinhala and English. Where LLMs are used in health settings, the decision to refuse is rarely made explicit or evaluated, and adaptation to the user is typically a fixed category.

The conceptual framework in Figure 2.1 links each observed problem to a design principle of HKAS: grounded generation answers the depth and accessibility problems; English-pivot query normalization addresses the Sinhala gap; evidence gating addresses hallucination risk; flexible audience categories address the diversity of users and the rigid-category problem; and a measurable evaluation addresses the absence of a framework.

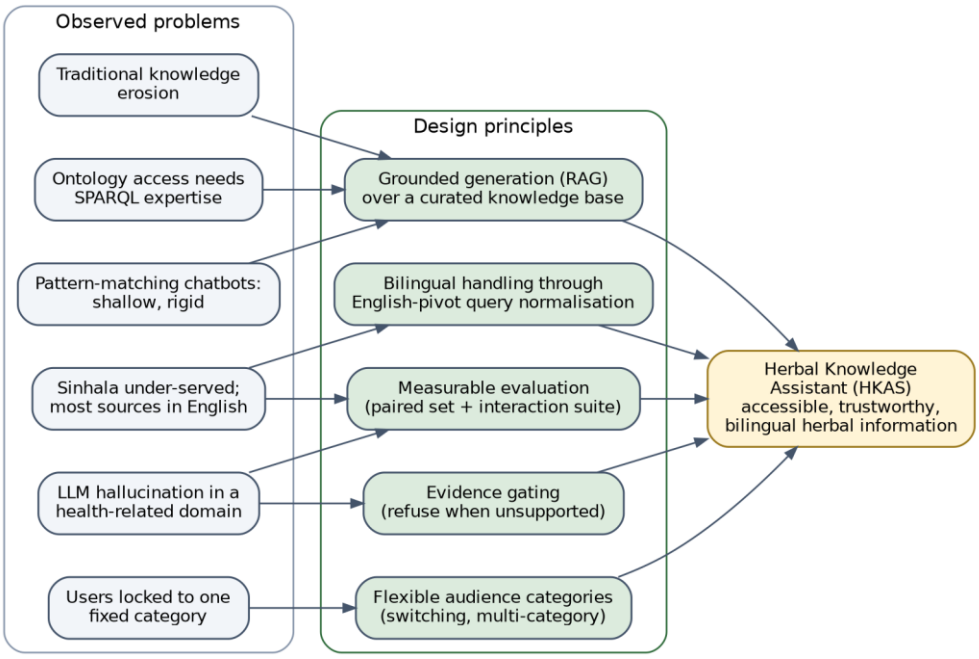


Figure 2.1 Conceptual framework linking observed problems to HKAS design principles

3 RESEARCH METHODOLOGY

3.1 Research Design

The work follows a design science approach [40]: an artefact (the HKAS pipeline and application) is built to address an identified problem, evaluated rigorously, and refined in response to the evaluation. Development was iterative. Each iteration added a capability, exercised it with test questions, and used the observed defects as input to the next iteration, as shown in Figure 3.1. Feedback from the project progress presentations, in particular the panel's request for more flexible user categories, drove one major iteration.

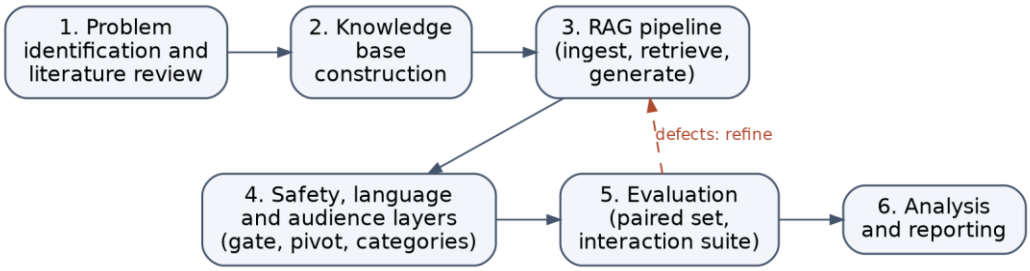


Figure 3.1 Research and development methodology

The study is mixed-methods. Quantitative measures (retrieval hits, refusals, scores, pass counts) are complemented by qualitative evidence (answer inspection, expert comments, usability feedback) and an error taxonomy.

3.2 Data Participants or Study Materials

3.2.1 Knowledge base

The study material consists of two plain-text files in the data folder. Medicinal Plants.txt holds 20 Sri Lankan medicinal plant entries. Each entry is a structured record with names (common, Sinhala and scientific), family, properties, medicinal usage, preparation, precautions and limitations, and evidence fields (evidence by use, evidence references and a scientific evidence level). disease_knowledge.txt holds plant-disease entries (description, typical symptoms and recommended action) that support the explainable quality assessment component of the project. Entries are separated by blank lines and range from 852 to 3,219 characters.

Plant entries were compiled from the published literature on Sri Lankan medicinal plants [2], with precaution and evidence fields added to give each entry safety context.

3.2.2 Evaluation sets

Suite A is a bilingual paired set: six plant questions asked in English and in Sinhala (twelve queries) covering neem (two questions), ginger, turmeric, gotukola and aloe vera. Pairing allows a direct comparison of language behavior for the same information need. The Sinhala questions are kept as a user would type them; reflecting the spelling variation of real input. Suite B is a flexible-interaction suite of eight tests, each with expected audience categories and required keywords, grouped as stays-home, switching, multi-category, multi-plant and explicit override. Both suites were run by evaluate_accuracy.py on the final version of the pipeline, and Chapter 6 reports that evaluation. Because model calls are not deterministic, the results indicate typical behavior and do not characterize run-to-run variation (Section 7.5). The questions are listed in Appendix A.

3.2.3 Participants

No formal participant study was carried out. Several users, including students and members of the public, tried the system informally and gave comments. Expert validation by Ayurvedic practitioners was not carried out and is listed as a limitation (Section 7.5) and as future work (Section 8.4).

3.3 Experimental or Study Procedure

The procedure has six stages.

Stage 1: baseline. A basic RAG pipeline was implemented: entries embedded, nearest entries retrieved and an LLM prompted to answer from them.

Stage 2: enrichment and ingestion. Entries were enriched with precaution and evidence fields, and ingestion was changed from size-based splitting to one chunk per entry.

Stage 3: gating and language handling. A cosine-distance threshold, a Sinhala lexical guard and a single-call query normalization step (context resolution, Sinhala-to-English translation, multi-plant splitting) were added.

Stage 4: flexible audiences. Following panel feedback, a four-category audience scheme with explicit, detected and home resolution, and multi-category answers, replaced a single fixed category.

Stage 5: evaluation. evaluate_accuracy.py ran Suite A and Suite B and saved the answers, sources, categories and flags. Manual scoring followed, and informal user comments were collected (Figure 3.2).

Stage 6: analysis and refinement. Observed issues were classified and refinements proposed, as reported in Chapters 5 to 7.

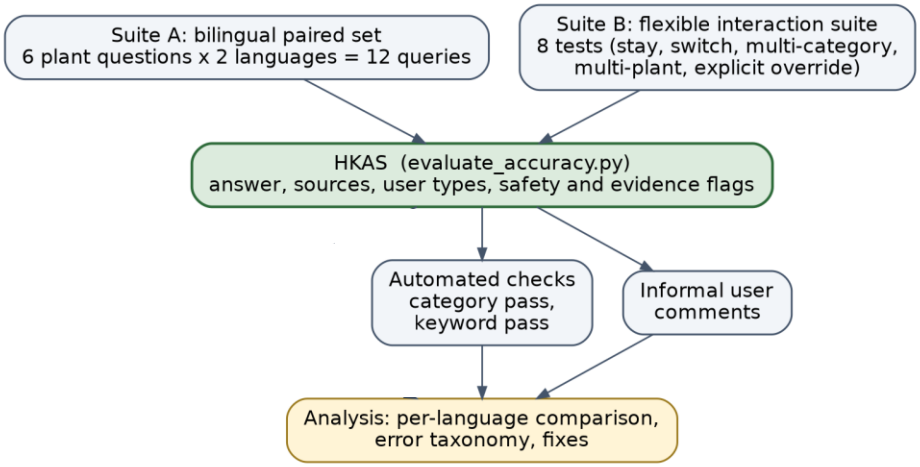


Figure 3.2 Evaluation procedure

3.4 Measures and Evaluation Metrics

Table 3.1 lists the measures. Retrieval and generation are examined separately so that a wrong answer can be attributed to the retriever, the generator or the knowledge base.

Table 3.1 Evaluation measures

Measure	Definition	Method
Retrieval hit	The expected plant entry appears among the sources returned	Sources list from the run
Wrong refusal	The system returns the refusal message although the knowledge base holds the answer	Sources empty and refusal text

Measure	Definition	Method
Language correctness	Answer language matches the question language	Inspection
Precaution present	The answer states a relevant precaution when the entry contains one	Inspection
Category pass	The returned audience categories exactly match the expected categories	Automated (Suite B)
Keyword pass	All required plant keywords appear in the answer	Automated (Suite B)

3.5 Data Analysis

Results are reported per language and overall, with each case inspected, because the sets are small. Observed issues are classified into a taxonomy: wrong refusal, retrieval miss, Sinhala generation error, language mismatch, label leak, heuristic misfire and flag semantics. Because model calls are not deterministic, a repeated-run measurement is left as future work.

3.6 Ethics Privacy Security and Responsible AI

Medical safety. HKAS provides educational information and is not a substitute for a qualified practitioner. Answers carry precautions where the knowledge base contains them, and the gate declines to answer rather than guess. The system does not diagnose conditions or recommend personalized dosages.

Privacy. Under the design, no personal user data is stored. Chat messages are retained only in session memory for conversational continuity.

Security. The API key is held in an environment variable and not in source control; the service validates and bounds user input to reduce prompt-injection and abuse risks of the kind catalogued for LLM applications [42]; HTTPS is required in any production deployment.

Responsible AI and cultural care. Traditional knowledge belongs to its communities of practice. Content is attributed to its sources, and the system is positioned as a means of preservation and education.

3.7 Reproducibility and Research Artefacts

The research artefacts are the knowledge base files (Medicinal Plants.txt and disease_knowledge.txt), ingestion.py, conversation_rag.py, the Flask service, the Flutter client, evaluate_accuracy.py with its outputs (evaluation_results.txt, evaluation_scores.csv) and check_retrieval.py. The vector store persisted in chroma_db_free/ and can be rebuilt by ingestion.py. The settings that determine behaviour are listed in Table 5.2.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Requirements were derived from the stakeholder analysis, the gaps in Chapter 1 and the panel feedback, and written in the style recommended for requirements engineering [43]. The stakeholders are the general public, Ayurvedic practitioners, students and researchers, each with different needs for depth and tone.

Table 4.1 Functional requirements

ID	Requirement	Priority
FR-01	Understand questions in English and Sinhala text	High
FR-02	Provide plant names (English, Sinhala, scientific) and family	High
FR-03	List medicinal uses and applications of a plant	High
FR-04	Explain preparation methods and parts used	High
FR-05	Always state the relevant precaution when the entry contains one	High
FR-06	State the evidence level for the use asked about and list references only from the knowledge base	High
FR-07	Refuse, in the language of the question, when the knowledge base does not support an answer	High
FR-08	Answer in the language of the question	High
FR-09	Support four audience categories (general public, practitioner, student, researcher) chosen on the client	High
FR-10	Switch category for one answer when the message signals another audience	Medium
FR-11	Combine up to two categories in one answer	Medium
FR-12	Answer questions about up to three plants without mixing their facts, and decline plants not in the knowledge base	Medium
FR-13	Maintain conversation context for follow-up questions (last three exchanges)	Medium
FR-14	Accept a plant name or disease label from other components as a short question and explain it	Medium

Table 4.2 Non-functional requirements

ID	Attribute	Requirement
NFR-01	Performance	Respond within an acceptable interactive time; support at least 10 concurrent test users
NFR-02	Accuracy	Answers must match the knowledge base, with no deliberate or accidental fabrication
NFR-03	Reliability	Degrade to a polite message, not an error, when retrieval or the LLM fails
NFR-04	Usability	Simple interface suitable for all age groups; bilingual interface text

ID	Attribute	Requirement
NFR-05	Security and privacy	No personal data stored; HTTPS in production; keys outside source control
NFR-06	Maintainability	Knowledge updated by editing the text files and re-ingesting
NFR-07	Portability	Backend runs locally with Python; client runs on Android

4.2 Component Architecture within the Complete Solution

Figure 4.1 shows HKAS in its deployment context. The Flutter client sends JSON requests to a Flask service. The service passes each request to the RAG engine, which resolves the audience categories, normalizes the search query, searches the ChromaDB collection, applies the distance gate and the Sinhala guard, builds the prompt and calls the chat model. Other components of the project can send a plant name or a disease label as a short question, which the engine explains.

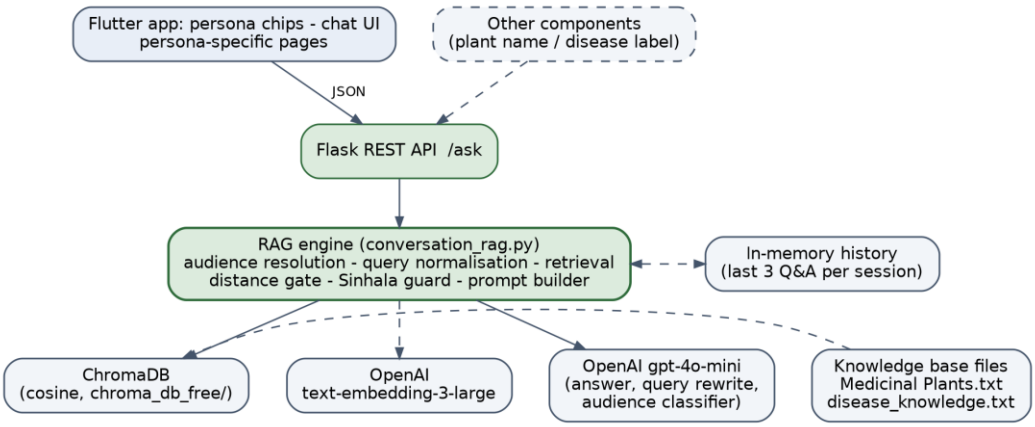


Figure 4.1 HKAS component architecture and interfaces

The interface with the other components is deliberately narrow: a short label in, a grounded answer out. This keeps the components independently developable and testable, which suits a four-person project with parallel work streams.

4.3 Design Alternatives and Decisions

Table 4.3 records the main design decisions, the alternatives considered and the reasons for each choice.

Table 4.3 Design decisions and alternatives

Decision	Alternatives	Rationale
One chunk per entry (split on the entry boundary)	Size-based splitting such as RecursiveCharacterTextSplitter	Entries vary from 852 to 3,219 characters; size-based splitting both cuts long entries and merges short neighbours, separating a use from its precaution or mixing two plants [23]
Dense retrieval only	Hybrid dense plus BM25	Simplest pipeline that meets the need at this scale; hybrid retrieval is future work
text-embedding-3-large with cosine distance	Smaller embedding model; Sinhala-specific encoders	Stronger multilingual semantic matching from a hosted service [8]; cosine distance gives a bounded scale (0 to 2) for a single threshold
English-pivot query normalisation in one LLM call	Embed Sinhala directly; separate rewrite and translate calls	One call resolves pronoun from history, translates Sinhala while preserving plant identity, and splits multi-plant questions, avoiding compounding errors between steps; retrieval then always happens in English
Distance threshold of 0.8 plus Sinhala lexical guard	Always answer; ask the LLM to judge support	A numeric gate is inspectable and tunable; self-judgement by the LLM is itself prone to hallucination [17]. The guard blocks Sinhala questions whose plant name never appears in the retrieved text
Audience resolution: explicit, then detected, then home	Fixed category per page	Meets the panel request for flexibility: users can switch category per message, combine two categories, or override with chips
gpt-4o-mini for answers, rewriting and classification	Larger models; open models [30]	Low cost and latency for a prototype [9]; a deterministic (temperature 0) instance is used for audience classification
In-memory conversation history	Firestore	Avoids personal-data storage in this prototype; history is lost on restart
ChromaDB	Vector databases	Lightweight, persistent and Python-native [7]
Flask backend and Flutter client	FastAPI	Minimal and well understood by the team [10]; Flutter is shared with other components [11]

4.4 Detailed Design

4.4.1 Knowledge base schema

Table 4.4 lists the fields. Keeping them in one entry means a single retrieved chunk carries the use, the preparation and the precaution together, so the generator cannot present a use without its safety context.

Table 4.4 Knowledge base entry fields

Field	Content
Names and taxonomy	Common name, Sinhala name, scientific name, family
Properties	Traditional properties (for example bitter, cooling) used in explanations
Medicinal usage	Conditions and uses in traditional practice
Preparation	Parts used and traditional preparation methods
Precautions/Limitations	Contraindications, cautions and when to seek professional advice
Evidence by Use	Level of support (traditional, laboratory, clinical) for each main use
Evidence References	Studies cited for the entry
Scientific Evidence Level	Overall summary of evidence strength
Disease entries	Description, typical symptoms and recommended action (disease_knowledge.txt)

4.4.2 Ingestion pipeline

Figure 4.2 shows ingestion. Both files are loaded, each file is split on the entry boundary, and each entry becomes exactly one chunk. The chunks are embedded with text-embedding-3-large and stored in a persisted ChromaDB collection created with cosine distance.

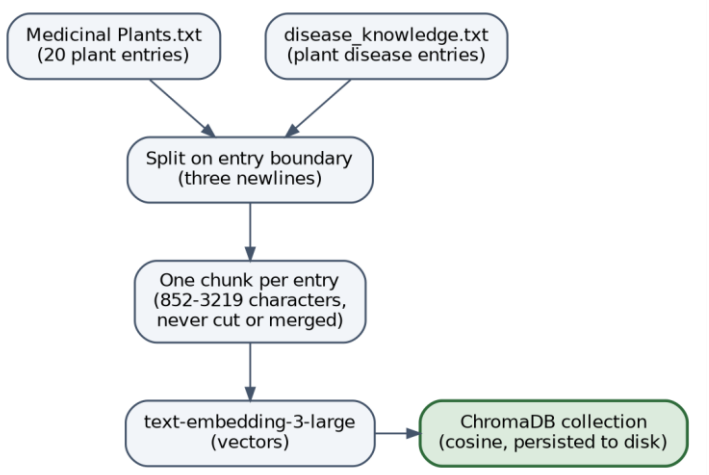


Figure 4.2 Ingestion pipeline

4.4.3 Query pipeline and evidence gating

Figure 4.3 shows how a question is processed. After the audience categories are resolved, one LLM call normalizes the search query. For a simple English first-turn question with no multi-plant or style cue, the call is skipped and the question is used as is. For Sinhala questions the call translates the question into English, keeps the

plant's identity, and prefers the common English and scientific names. For questions about several plants, it returns up to three queries, one per plant.

Retrieval then runs once per query, with k = 5 for a single query and k = 3 per query for several and keeps chunks whose cosine distance is at most 0.8, merged, de-duplicated and sorted by distance. If nothing remains, the system returns a refusal in the language of the question with no sources. For Sinhala questions a further guard requires at least one Sinhala word of the question to appear in the retrieved text; otherwise, the system refuses. Otherwise, the prompt is built and the model answers.

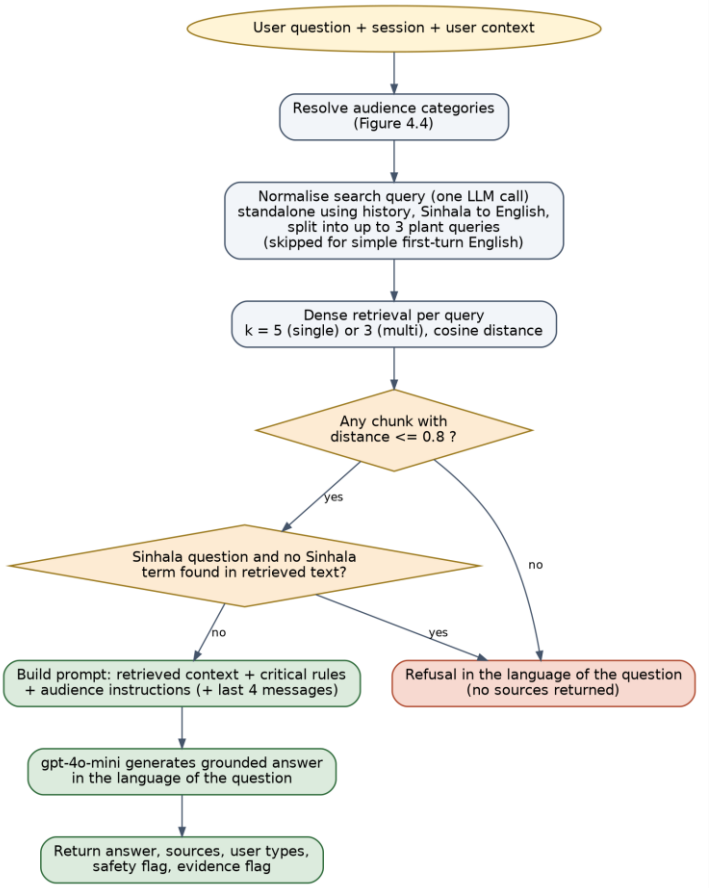


Figure 4.3 Query pipeline with evidence gating and the Sinhala guard

4.4.4 Audience resolution

Each user type is a home category tied to a page of the client. Figure 4.4 shows how the categories for one answer are chosen. Explicit chip choices win and are sticky because the client resends them. Otherwise, a deterministic classifier call (temperature 0, up to two categories) detects an audience signal in the message, for

example a request for evidence (researcher), for an explanation of why (student), for preparation or dosage (practitioner) or for simple words (general public). The detected category applies to that answer only. If there is no signal, the home category is used, with the general public as the default.

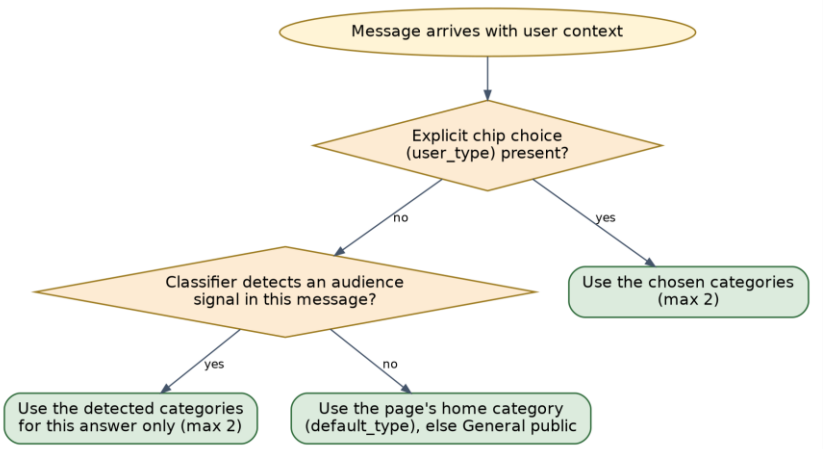


Figure 4.4 Resolution of audience categories for one message

Table 4.5 summarizes the four categories. The same retrieved evidence is used for all; only the depth, vocabulary and emphasis differ. When two categories apply, one answer with a short labelled section per category is produced.

Table 4.5 Audience categories and response style

Category	Need	Response style
General public	Simple, practical, safe information	Three to five short sentences in everyday words; one plain precaution sentence; no evidence levels or Latin names unless asked
Practitioner	Quick professional reference	Clinical terminology; plant part and preparation, indications, precautions, one line on evidence; dosage only if in the context
Student	Learning and understanding	Names the plant with scientific name and family; explains why each use applies through the named property; ends with key points
Researcher	Evidence and detail	Precise prose; for each use states whether support is traditional, laboratory or clinical; names gaps and toxicity; lists only the references in the context

4.4.5 Prompt design

The generation prompt states critical rules: answer only about the plants named in the question; cover several plants separately and never mix their facts; say so when the

context lacks a requested plant while still answering for the others; answer in the language of the question; switch completely when the topic changes; always mention a precaution if the context has one; state the evidence level for the use asked about and never invent studies; write traditional claims as traditional; avoid markdown and field names as headings; and refuse if the context does not match the question. The audience instruction is appended, followed by the question and the retrieved context. The last four messages of history are included. Because the context contains only the entries that passed the gate, it stays short, which avoids the degradation seen with long contexts [36].

Short English inputs of at most six words without a question mark, such as a plant name or a disease label sent by another component, are expanded in the prompt into a full question asking what it is, its typical symptoms and whether affected leaves can still be used medicinally. This supports the integration with the quality assessment component.

4.4.6 API and client design

The Flask service exposes the /ask endpoint, which accepts the question, a session identifier and a user context holding the home category (default_type) and any explicit choice (user_type), and returns the answer, the sources, the list of categories used, and two flags: safety_context_available (a retrieved entry contains Precautions/Limitations) and evidence_references_available.

The Flutter client keeps the persona chip selector. Selecting a category opens a page for that user type with a distinct layout and the chat interface, and the page's category is sent as default_type.

4.5 Standards and Professional Practice

The API follows REST conventions with JSON payloads. Security practice follows the OWASP guidance for LLM applications [42], covering prompt injection, sensitive information disclosure and over-reliance. Privacy design follows Sri Lanka's Personal Data Protection Act [41]. Source code is version controlled, secrets

are kept in environment variables, and the interface text and color contrast follow accessibility good practice.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

The backend is written in Python and developed on Windows. Table 5.1 lists the technologies and the reason each was selected, and Table 5.2 records the configuration values that determine system behavior.

Table 5.1 Technology stack

Layer	Technology	Purpose
Language	Python 3	RAG engine, API, evaluation scripts
Orchestration	LangChain [6]	Document loading, vector store access, model calls
Vector store	ChromaDB [7]	Persistent embeddings in chroma_db_free/ (cosine distance)
Embeddings	OpenAI text-embedding-3-large [8]	Semantic representation of entries and English search queries
Generation	OpenAI gpt-4o-mini [9]	Answers, query normalisation and audience classification
API	Flask [10]	REST service
Client	Flutter / Dart [11]	Mobile chat interface, persona chips and pages
Evaluation	Python scripts and CSV	Scripted runs and scoring sheets

Table 5.2 Configuration parameters

Parameter	Value	Note
Distance threshold	0.8	Cosine distance; chunks farther than this are discarded
Retrieval size k	5 (single query), 3 per query (multi)	Up to three queries per question
Answer model	gpt-4o-mini, temperature 0.15, max_tokens 900	Frequency penalty 0.9, presence penalty 0.7
Audience classifier	gpt-4o-mini, temperature 0, max_tokens 60	Returns up to two categories as JSON
Maximum categories per answer	2	MAX_PERSONAS
History	Last 4 messages in the prompt; last 6 stored	Three question-answer pairs
Embedding model	text-embedding-3-large	Used for ingestion and queries

5.2 Component Implementation

5.2.1 Knowledge base and ingestion

The knowledge base began as basic plant descriptions and was enriched with precaution and evidence fields. The enrichment was motivated by the safety objective: a question about use should be answerable together with the relevant caution, and the evidence level for that use. The disease entries were added for the quality assessment component and share the same vector store.

Ingestion loads every .txt file in the data folder, splits it on the entry boundary and keeps each entry as one chunk, as in the excerpt below from ingestion.py. A size-based splitter was rejected because it cut long entries and merged short neighbours.

```
entries = doc.page_content.split("\n\n\n")
for entry in entries:
    entry = entry.strip()
    if entry:
        chunks.append(Document(page_content=entry, metadata=doc.metadata))


vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=OpenAIEmbeddings(model="text-embedding-3-large"),
    persist_directory="chroma_db_free",
    collection_metadata={"hnsw:space": "cosine"})
```

5.2.2 Query pipeline, gating and audience resolution

The normalisation call is a single prompt that returns only search query text, one line per plant, up to three. Retrieval applies the distance threshold per result, as in the excerpt below from conversation_rag.py.

```
for q in queries:
    for doc, score in vectorstore.similarity_search_with_score(q, k=k):
        if score <= DISTANCE_THRESHOLD and doc.page_content not in seen:
            seen.add(doc.page_content)
            merged.append((doc, score))
return sorted(merged, key=lambda x: x[1])
```

If the merged list is empty, ask_question returns the refusal. For Sinhala questions the guard checks that a Sinhala word of the question occurs in the combined retrieved text, again refusing otherwise.

5.2.3 Persona-aware answer generation

The PERSONA_INSTRUCTIONS mapping provides four output policies. A general-public answer is intended to be short and accessible. A practitioner answer emphasizes the part and preparation, indications, contraindications, and evidence caveats; doses should not be invented when absent from the source. A student answer includes names and explanatory relationships and ends with memorable key points. A researcher answer is designed to distinguish traditional claims from experimental and clinical evidence and reproduce only citations present in the retrieved source.

The resolver implements a priority order: explicit user selection, current-message intent, and current page default. Combining categories yields separate labelled sections within one answer. The constant MAX_PERSONAS limits output to two audience types and MAX_SEARCH_QUERIES limits retrieval expansion to three queries. These fixed limits make the behaviour testable, although broad user requests can exceed the configured scope.

The excerpt below shows the priority order. The client resends explicit choices, so they persist, whereas a detected category affects one answer.

```
explicit = user_context.get("user_type")
if explicit and explicit != "auto":
    explicit = [explicit] if isinstance(explicit, str) else list(explicit)
    explicit = [p for p in explicit if p in ALLOWED_PERSONAS][:MAX_PERSONAS]
    if explicit:
        return explicit
detected = _detect_personas(question)
if detected:
    return detected
return [home]
```

5.2.4 API and client

The Flask application exposes /ask, validates the input, calls ask_question with the question, session and user context, and returns the JSON result. Errors from retrieval or the model are caught and returned as a polite message, in line with NFR-03.

The client presents a persona chip selector; choosing a category opens a page for that user type with a distinct layout and the chat interface.

5.3 Verification and Software Testing

Testing combined scripted evaluation of the whole pipeline, with its outputs inspected by hand, and usability testing. Table 5.3 lists representative test cases mapped to requirements; the observed outcomes are reported once, in Chapter 6.

Table 5.3 Representative test cases

ID	Scenario	Input	Expected result
TC-01	English factual query	What is neem used for?	Grounded answer with caution
TC-02	Sinhala factual query	Ginger question in Sinhala	Grounded answer in Sinhala
TC-03	Sinhala neem query	Neem question in Sinhala	Grounded answer
TC-04	Plant and disease collision	How to prepare neem for skin diseases?	Preparation from the plant entry
TC-05	Answer language	Sinhala neem skin question	Answer in Sinhala
TC-06	Stays in home category	Student page, plain question (F1, F3)	Student category
TC-07	Category switching	Is there scientific evidence for it? (F2); Explain why ginger helps (F4)	Researcher; student
TC-08	Multi-category	Preparation and evidence for turmeric (F5)	Practitioner and researcher
TC-09	Multi-plant	Compare ginger and turmeric for cold (F6)	Both plants, no mixing
TC-10	Partly unknown plant	Ginger and dragon fruit (F7)	Answer ginger, decline dragon fruit
TC-11	Explicit override	Chips practitioner and student (F8)	Both categories
TC-12	Follow-up with pronoun	Is there scientific evidence for it? (F2)	Resolved to neem

ID	Scenario	Input	Expected result
TC-13	Out-of-knowledge-base query	Non-plant or unknown question	Refusal
TC-14	Concurrency and latency	10 concurrent requests	Answered in acceptable time

5.4 Deployment and Operation

The prototype runs locally: the Flask service is started on port 5000 with the OpenAI API key supplied through an environment variable, the persisted ChromaDB collection is loaded from chroma_db_free/ (or rebuilt from the knowledge base), and the Flutter client connects to the service address. For a production deployment the service would be placed behind HTTPS on a cloud host, with logging, rate limiting and a persistent conversation store.

6 RESULTS AND VALIDATION

6.1 Research Results

This section reports the evaluation of Suites A and B. Table 6.1 gives the composition of the suites. Retrieval hits, refusals, language and category results are objective counts from the saved outputs.

Table 6.1 Evaluation suites

Suite	Content	Queries
A: bilingual paired set	Neem (two questions), ginger, turmeric, gotukola, aloe vera; English and Sinhala	12
B: flexible-interaction suite	Stays home (2), switching (2), multi-category (1), multi-plant (2), explicit override (1) in 6 sessions	8

Table 6.2 Outcome by language, Suite A

Measure	English	Sinhala
Answer in the language of the question	6 of 6	5 of 6
Relevant precaution stated in the answer	5 of 5 answered from a plant entry	6 of 6

Retrieval hits and misses for each pair are given in Table 6.3. English answers were well grounded: the correct entry was retrieved for five of six questions, and every English answer based on a plant entry stated a relevant precaution. The one miss was the question on preparing neem for skin diseases, where the retrieved entries were plant-disease records.

The English pivot gave strong Sinhala retrieval: all six Sinhala questions retrieved the correct plant entry, including the question typed without a space (question 4), and none was refused. Every Sinhala answer stated the relevant precaution from the entry, and five of six were written in Sinhala. For five of the six pairs the Sinhala and English answers agreed in substance, with only minor differences.

Table 6.3 Per-query results, Suite A

Pair	Plant	English	Sinhala	Observation
P1	Neem	Hit; 2	Hit; 1	Sinhala answer adds one use not in the English answer and renders a toxicity caution as infection
P2	Neem (skin)	Miss; 0	Hit; 1	English retrieved disease entries only; Sinhala answered in English and included audience labels
P3	Ginger	Hit; 2	Hit; 1	Uses and cautions (gallstones, blood-thinning medicines) correct; one garbled phrase
P4	Turmeric	Hit; 2	Hit; 1	Uses and skin-irritation caution correct; some terms mistranslated; stray closing sentence
P5	Gotukola	Hit; 2	Hit; 1	Memory use and caution correct; one English word mixed in and a stray closing sentence; English answer includes audience labels
P6	Aloe vera	Hit; 2	Hit; 1	Latex caution correct; some uses imprecisely phrased (brain diseases, fruit); stray closing sentence

Table 6.3 shows that the gate admitted evidence for all twelve queries and refused none, and the Sinhala guard blocked none of the six Sinhala questions. The retrieval miss for the English skin-disease question shows a different behavior: the distance gate admitted three plant-disease entries because they were semantically close to a question mentioning neem and diseases, so it passed related evidence. The model nevertheless said it had no information on preparing neem for skin diseases, which is the intended conservative. Separating plant and disease entries by metadata is the proposed refinement.

Regarding the safety flag, safety_context_available was true for every query that retrieved a plant entry and false only for the English skin-disease question. It reflects retrieved context, precaution statements were therefore checked in the answer text (Table 6.2). The comparison of the gate against an ungated baseline and the behavior on out-of-knowledge-base questions are listed as future work.

All eight tests passed both automated checks: the returned categories matched the expected categories (8 of 8) and all required keywords appeared (8 of 8). The pronoun in F2 was resolved to neem from the previous turn, a student page switched to the researcher style for one answer and returned to student style on the next question (F3), the two categories of F5 were combined in one answer, and an explicit choice overrode the page type in F8. Manual inspection added three observations that

the automated checks cannot see: in F6 the answer mentions affected leaves because the short-label rule expanded the question as a disease label; in F5 and F8 the answer contains the text of the audience instruction; and F7 needs a manual check that dragon fruit was declined, since only the first 200 characters were saved. The keyword check confirms presence only and cannot detect fact mixing between plants.

6.2 Product and Technical Results

Table 6.4 shows the delivered features against the requirements. Response time was not measured in the evaluation reported here.

Table 6.4 Delivered functionality

Requirement	Feature	Status
FR-01, FR-08	Bilingual question answering in the language of the question	Delivered; Sinhala phrasing being refined
FR-05, FR-06	Precautions, evidence levels and references from the knowledge base	Delivered
FR-07	Refusal when unsupported (distance gate, Sinhala guard)	Delivered; no answerable question refused in Suite A
FR-09 to FR-11	Four categories, per-message switching, multi-category answers	Delivered
FR-12	Multi-plant questions (up to three)	Delivered
FR-13	Conversation context (three exchanges, in memory)	Delivered
FR-14	Plant and disease labels from other components	Interface defined

6.3 User or Stakeholder Validation

During prototype development, the functioning Flutter chatbot was explored informally by friends and peers, who commented verbally on answers. This feedback served as iterative developer-facing input, particularly for interface behavior and intelligibility.

6.4 Integration Results

End-to-end integration with the other components was not part of the evaluation reported here. The interface through which plant names and disease labels reach HKAS is described in Section 4.2 and 4.4.5, and a full demonstration depends on the completion of the other components.

6.5 Summary of Findings

The evaluation established the following findings. English grounded answering works well: the correct plant entry was retrieved for five of six English questions, and every English answer based on a plant entry stated a relevant precaution. Sinhala retrieval through the English pivot is effective: all six Sinhala questions retrieved the correct plant entry and none was refused; five of six were answered in Sinhala. Sinhala answers are grounded and carry the precaution, and their phrasing is the main area for refinement: core uses and cautions agreed with the English answers, while phrasing errors, one added use and a stray closing sentence on three answers (Table 6.3) point to prompt and penalty settings that can be tuned. Plant and disease entries share one vector space: a question combining neem and diseases retrieved disease entries only, which metadata separation can address. Flexible audience resolution works as designed: all eight interaction tests returned the expected categories, including multi-category answers, explicit override and pronoun resolution across turns. Two prompt-level details need tidying: audience instruction text appeared in some answers, and the short-label rule misfired on one comparison question. The flags describe retrieved context rather than the answer text, and the evaluation sets are small with a keyword check that cannot detect fact mixing; out-of-knowledge-base behaviour, fact mixing and latency remain to be measured systematically.

7 DISCUSSION

7.1 Interpretation of Findings

RQ1 (grounded accuracy). The findings support the use of RAG over a curated base: five of six English questions retrieved the correct entry and produced answers that matched the entry's uses and cautions, and all six Sinhala questions retrieved the correct entry and stated its precaution. The one English miss was caused by corpus design, not by the generator. Sinhala answers were scored as partly correct because of phrasing-level errors, not because of wrong retrieval. RQ2 (gate behaviour). The gate behaved conservatively where it mattered: it refused no answerable question in Suite A and the Sinhala guard blocked none. Its limit is that it measures semantic closeness, not usefulness, so in the English skin-disease case it admitted loosely related disease entries. Whether it blocks truly unsupported questions has not yet been measured systematically. RQ3 (Sinhala versus English). The English pivot makes retrieval language-independent (six of six for Sinhala against five of six for English). RQ4 (flexible audiences). Supported on the suite: eight of eight tests were routed as expected. The panel's concern, that users are locked to one category, is addressed by the explicit, detected and home priority order, with multi-category answers and per-message switching. The cost is occasional label leaks and longer answers when two categories are detected for a plain question. Two lessons concern engineering. A convenience flag is not a safety guarantee, so the final text must be evaluated, as the failure points described for RAG systems also suggest [25]. And because model calls are not deterministic, a single run indicates typical behaviour but not its variation, so repeated runs are planned.

7.2 Comparison with Literature Baselines and Existing Solutions

Compared with the ontology of Jeyananthan et al. [12], HKAS removes the need for SPARQL and allows free-form questions, at the cost of structured guarantees. Compared with the AIML and intent chatbots [13], [14], it handles differently phrased questions semantically, draws on richer entries, and adds precautions and evidence statements instead of fixed replies. Compared with general-purpose LLM

assistants, it limits answers to a curated source and refuses when evidence is weak. Published accuracy figures for earlier chatbots, such as 96% for the intent classifier in [14], measure matching to predefined intents and are not comparable with the open-ended answers evaluated here. Unlike those systems, HKAS is evaluated on open-ended answers, with its refinement areas documented in Section 6.5.

7.3 Research Contribution

The research contributions are: (1) a bilingual RAG design for Sri Lankan herbal knowledge based on entry-level chunks; (2) single-call query normalisation that combines context resolution, Sinhala-to-English translation and multi-plant splitting; (3) an inspectable evidence gate (distance threshold and language guard) that turns weak evidence into refusal; (4) flexible audience resolution with explicit, detected and home categories and multi-category answers, responding to panel feedback; (5) evidence-aware prompting that requires precautions and forbids invented references; and (6) an evaluation protocol and error taxonomy for bilingual herbal assistants, addressing Gap 5.

7.4 Product Contribution

The product contribution is a working prototype: a Flask API and Flutter client, a curated knowledge base of 20 plants with precaution and evidence fields together with plant-disease entries, and an evaluation toolkit. It can be used by the public, students, practitioners and researchers, and forms the explanatory layer of the wider R26-IT-021 ecosystem.

7.5 Limitations and Threats to Validity

Knowledge base size. Only 20 plants are covered, against roughly 1,400 species in use, so real users would meet many refusals.

Single scripted run. Model calls are not deterministic, and each scripted question ran in a fresh session without client context; repeated runs would show the variation of the results and are listed as future work (Section 8.4).

Small evaluation sets. Twelve paired queries and eight interaction tests cannot support statistical claims and may not generalise.

No negative test set yet. Behaviour on out-of-knowledge-base questions, and the effect of the gate against a baseline, have not been measured in this evaluation.

Model dependence. Behaviour depends on proprietary models that can change over time, cost money and receive user queries.

Heuristic settings. The 0.8 threshold, the penalties and the short-label rule were set empirically and may need retuning as the knowledge base grows.

7.6 Implications

For practice, the work shows that a modest RAG system with explicit refusal and mandatory precautions is designed to give safer access to traditional knowledge than a general chatbot, provided that the weakest language path is tested separately. For research, it suggests that an English pivot is an inexpensive way to support a low-resource language but moves the risk to generation, so answer text in that language must be evaluated directly. For communities, a searchable, bilingual record contributes to preserving knowledge held by elderly practitioners, which supports Sustainable Development Goals 3 (Good Health and Well-being) and 4 (Quality Education) [45].

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

This thesis set out to give the Sri Lankan public, students, practitioners and researchers conversational, bilingual and grounded, precaution-aware access to medicinal plant knowledge, and delivered HKAS, a RAG assistant with an evidence gate, English-pivot query normalisation and flexible audience categories. The evaluation showed grounded English answers, correct Sinhala retrieval and correct audience routing; the findings (Section 6.5) and their interpretation (Section 7.1) identify the refinements to pursue next.

8.2 Achievement of Objectives

Objectives RO1 to RO3, RO5 and RO6 were achieved: the 20-plant knowledge base with precaution and evidence fields (Section 3.2.1), the RAG pipeline (Section 5.2), the evidence gate, flexible audiences and multi-plant questions and the evaluation of two suites (Chapter 6). Objective RO4 was achieved, with Sinhala retrieval matching English retrieval and Sinhala phrasing still being refined. Objective RO7 is met at interface level (Section 5.3), with an end-to-end demonstration depending on the other components (Section 6.4). Formal expert and usability evaluation are planned (Section 8.4).

8.3 Individual Research and Product Contributions

The individual research contributions are the query normalisation and gating design, the flexible audience resolution and the evaluation protocol with its error taxonomy. The product contributions are the enriched knowledge base, the RAG engine, the Flask service, the Flutter client and the evaluation toolkit, as detailed in Sections 7.3 and 7.4.

8.4 Future Directions

Sinhala quality: Test lower generation penalties for Sinhala, replace the lexical guard with a plant-name alias list, and have Sinhala answers reviewed by native speakers.

Retrieval: Separate plant and disease entries by metadata, and evaluate hybrid dense plus lexical retrieval with re-ranking.

Evaluation: Add out-of-knowledge-base and multi-plant fact-mixing tests, a larger paired set, repeated runs, multiple raters and automated RAGAS runs.

Knowledge: Expand the base towards the commonly used species, with per-use evidence and expert review.

Product: Persistent conversations with authentication, voice input in Sinhala.

Integration: Connect fully with quality assessment and cultivation recommendation so that one image leads to knowledge, quality and economic guidance.

REFERENCES

[1] World Bank, "Medicinal Plants: Conservation and Sustainable Use in Sri Lanka," Indigenous Knowledge (IK) Notes, no. 66, Washington, DC, USA, 2004. [Online]. Available: https://hdl.handle.net/10986/10771

[2] S. N. Arseculeratne, A. A. L. Gunatilaka, and R. G. Panabokke, "Studies on medicinal plants of Sri Lanka. Part 14: Toxicity of some traditional medicinal herbs," J. Ethnopharmacology, vol. 13, no. 3, pp. 323-335, 1985, doi: 10.1016/0378-8741(85)90078-9.

[3] World Health Organization, WHO Global Report on Traditional and Complementary Medicine 2019. Geneva, Switzerland: WHO, 2019.

[4] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459-9474.

[5] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.

[6] H. Chase, "LangChain," 2022. [Online]. Available: https://github.com/langchain-ai/langchain

[7] Chroma, "Chroma: The AI-native open-source embedding database." [Online]. Available: https://www.trychroma.com

[8] OpenAI, "New embedding models and API updates," Jan. 2024. [Online]. Available: https://openai.com/index/new-embedding-models-and-api-updates/

[9] OpenAI, "GPT-4o mini: Advancing cost-efficient intelligence," Jul. 2024. [Online]. Available: https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/

[10] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com

[11] Google, "Flutter documentation." [Online]. Available: https://docs.flutter.dev

[12] P. Jeyananthan, E. Y. A. Charles, and D. A. S. Atukorale, "Ontology development for Sri Lankan medicinal plants: A knowledge representation," Sri Lanka Association for Artificial Intelligence (SLAAI), 2015.

[13] A. D. A. D. S. Jayalath, P. V. D. Nadeeshan, T. G. A. G. D. Amarawansha, H. P. Jayasuriya, and D. P. Nawinna, "Ayurvedic knowledge sharing platform with Sinhala virtual assistant," in Proc. 2019 Int. Conf. Advancements in Computing (ICAC), Malabe, Sri Lanka, 2019, pp. 220-225, doi: 10.1109/ICAC49085.2019.9103413.

[14] L. P. D. S. Senevirathne, D. P. D. S. Pathirana, A. L. Silva, M. G. S. R. Dissanayaka, D. P. Nawinna, and D. Ganegoda, "Mobile-based assistive tool to identify & learn medicinal herbs," in Proc. 2nd Int. Conf. Advancements in Computing (ICAC), Malabe, Sri Lanka, 2020, pp. 97-102, doi: 10.1109/ICAC51239.2020.9357247.

[15] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, art. 248, pp. 1-38, 2023.

[16] K. Singhal et al., "Large language models encode clinical knowledge," Nature, vol. 620, pp. 172-180, 2023.

[17] A. J. Thirunavukarasu, D. S. J. Ting, K. Elangovan, et al., "Large language models in medicine," Nature Medicine, vol. 29, pp. 1930-1940, 2023, doi: 10.1038/s41591-023-02448-8.

[18] U. H. Khan, M. H. Khan, and R. Ali, "Large language model based educational virtual assistant using RAG framework," Procedia Computer Science, vol. 252, pp. 905-911, 2025, doi: 10.1016/j.procs.2025.01.051.

[19] K. Sundar, E. Manohar, K. Vijay, and S. Prakash, "Revolutionizing assessment: AI-powered evaluation with RAG and LLM technologies," in Proc. 2nd Int. Conf. Self Sustainable Artificial Intelligence Systems (ICSSAS), Erode, India, 2024, pp. 43-48, doi: 10.1109/ICSSAS64001.2024.10760285.

[20] F. R. Shafi and M. A. Hossain, "LLM-Therapist: A RAG-based multimodal behavioral therapist as healthcare assistant," in Proc. IEEE GLOBECOM, Cape Town, South Africa, 2024, pp. 2129-2134, doi: 10.1109/GLOBECOM52923.2024.10901139.

[21] A.-M. Tanasa, S.-V. Oprea, and A. Bara, "Designing an architecture of a multi-agentic AI-powered virtual assistant using LLMs and RAG for a medical clinic," Electronics, vol. 15, no. 2, art. 334, 2026, doi: 10.3390/electronics15020334.

[22] H. Chia, A. I. Oliveira, and P. Azevedo, "Implementation of an intelligent virtual assistant based on LLM models for irrigation optimization," in Proc. 8th Int. Young Engineers Forum on Electrical and Computer Engineering (YEF-ECE), Caparica, Portugal, 2024, pp. 94-100, doi: 10.1109/YEF-ECE62614.2024.10624819.

[23] M. Pondel et al., "AI tools for knowledge management - Knowledge base creation via LLM and RAG for AI assistant," in Emerging Challenges in Intelligent Management Information Systems, M. Hernes, J. Watrobski, and A. Rot, Eds. Cham: Springer, 2024, vol. 1218, pp. 1-15, doi: 10.1007/978-3-031-78468-2_1.

[24] S. Ranathunga and N. de Silva, "Some languages are more equal than others: Probing deeper into the linguistic disparity in the NLP world," in Proc. 2nd Conf. Asia-Pacific Chapter of the ACL and 12th Int. Joint Conf. Natural Language Processing (AACL-IJCNLP), 2022, pp. 823-848.

[25] S. Barnett, S. Kurniawan, S. Thudumu, Z. Brannelly, and M. Abdelrazek, "Seven failure points when engineering a retrieval augmented generation

system," in Proc. IEEE/ACM 3rd Int. Conf. AI Engineering - Software Engineering for AI (CAIN), 2024, arXiv:2401.05856.

[26] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," arXiv:2309.15217, 2023.

[27] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 5998-6008.

[28] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171-4186.

[29] T. B. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 1877-1901.

[30] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," arXiv:2307.09288, 2023.

[31] W. X. Zhao et al., "A survey of large language models," arXiv:2303.18223, 2023.

[32] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 6769-6781.

[33] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982-3992.

[34] A. Neelakantan et al., "Text and code embeddings by contrastive pre-training," arXiv:2201.10005, 2022.

[35] A. Conneau et al., "Unsupervised cross-lingual representation learning at scale," in Proc. 58th Annu. Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 8440-8451.

[36] N. F. Liu et al., "Lost in the middle: How language models use long contexts," Transactions of the Association for Computational Linguistics, vol. 12, pp. 157-173, 2024.

[37] A. Abd-alrazaq et al., "Large language models in medical education: Opportunities, challenges, and future directions," JMIR Medical Education, vol. 9, art. e48291, 2023, doi: 10.2196/48291.

[38] S. D'Urso, B. Martini, and F. Sciarrone, "A novel LLM architecture for intelligent system configuration," in Proc. 28th Int. Conf. Information Visualisation (IV), Coimbra, Portugal, 2024, pp. 326-331, doi: 10.1109/IV64223.2024.00063.

[39] A. Joly et al., "Interactive plant identification based on social image data," Ecological Informatics, vol. 23, pp. 22-34, 2014.

[40] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," MIS Quarterly, vol. 28, no. 1, pp. 75-105, 2004.

[41] Personal Data Protection Act, No. 9 of 2022, Parliament of the Democratic Socialist Republic of Sri Lanka, 2022.

[42] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2025. [Online]. Available: https://genai.owasp.org

[43] ISO/IEC/IEEE, "Systems and software engineering - Life cycle processes - Requirements engineering," ISO/IEC/IEEE 29148:2018, 2018.

[44] ISO/IEC, "Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models," ISO/IEC 25010:2011, 2011.

[45] United Nations, Transforming Our World: The 2030 Agenda for Sustainable Development, A/RES/70/1, 2015.

GLOSSARY

Chunk – A unit of text stored and retrieved as one item. In this work, one plant or disease entry.

Cosine distance – A measure of dissimilarity between two vectors, from 0 (identical direction) to 2 (opposite). Smaller means more similar.

Dense retrieval – Retrieval that compares vector representations of the query and documents rather than matching keywords.

Embedding – A numerical vector representing the meaning of a text, so that similar texts have nearby vectors.

Evidence gate – A rule that discards retrieved chunks beyond a distance threshold and triggers refusal when none remain.

Persona (audience category) – A user type (general public, practitioner, student or researcher) used to adapt the tone and depth of answers.

Query normalization – Rewriting a user question as a standalone English search query, using history and translation as needed.

Retrieval-Augmented Generation – A technique in which a language model generates answers from passages retrieved from an external knowledge base.

Vector store – A database that stores embeddings and returns the nearest ones to a query vector.

APPENDICES

Appendix A Research Instruments or Extended Results

This appendix lists the evaluation instrument and supporting material referred to in Chapters 3, 5 and 6.

Table A 1 Suite A: question pairs

Pair	English question	Sinhala question
P1	What is neem used for?	කොහොඹ භාවිතා කරන්නේ කුමක් සඳහාද?
P2	How to prepare neem for skin diseases?	සමේ රෝග සඳහා කොහොඹ භාවිතා කරන්නේ කෙසේද?
P3	What is ginger used for?	ඉඟුරු භාවිතා කරන්නේ කුමක් සඳහාද?
P4	What is turmeric used for?	කහ භාවිතාකරන්නේ කුමක් සඳහාද?
P5	Is gotukola good for memory?	ගොටුකොල මතකයට හොඳද?
P6	What is aloe vera used for?	කෝමාරිකා භාවිතා කරන්නේ කුමක් සඳහාද?

Table A 2 Suite B: flexible-interaction tests

ID	Group	Home category	Question	Expected categories
F1	Stays home	student	What is neem used for?	student
F2	Switching	student	Is there scientific evidence for it?	researcher
F3	Stays home	student	What is turmeric used for?	student
F4	Switching	general public	Explain why ginger helps with nausea	student
F5	Multi-category	general public	Give me the preparation method and the scientific evidence for turmeric	practitioner, researcher
F6	Multi-plant	practitioner	Compare ginger and turmeric for cold	practitioner
F7	Multi-plant	researcher	Tell me about ginger and dragon fruit	researcher
F8	Explicit	general public (chips: practitioner, student)	What is ginger used for?	practitioner, student

Appendix B Academic Integrity and AI Use Disclosure

Generative AI tools were used as an assistance mechanism during thesis drafting, language refinement, formatting support, and selected software-development discussions. The researcher reviewed and revised generated material, checked

technical descriptions against the implemented component, and retained responsibility for the final wording, citations, code, data interpretation, and conclusions. AI-generated text was not treated as a medical or Ayurvedic authority. Plant information, disease information and cited academic claims were grounded in the identified health-information resources, classical Ayurvedic sources, research literature, project artefacts, and implementation evidence. No confidential credentials or authentication secrets are intentionally included in the thesis.